

Содержание

1. Зачем нужен этот документ	5
2. Общая постановка задачи	5
3. Что такое нейронная сеть	5
3.1. Базовая идея	5
3.2. Искусственный нейрон	6
3.2.1. Интуиция	6
3.2.2. Маленький пример	6
4. Что такое слой и почему одной линейности недостаточно	7
4.1. Линейный слой	7
4.2. Почему нужна нелинейность	7
4.3. Пример	7
5. Функции активации	7
5.1. Что это такое	7
5.2. Sigmoid	8
5.2.1. Интуиция	8
5.2.2. Плюсы	8
5.2.3. Минусы	8
5.3. Tanh	8
5.3.1. Интуиция	8
5.3.2. Минус	8
5.4. ReLU	8
5.4.1. Интуиция	8
5.4.2. Почему ReLU удобна	8
5.4.3. Пример	9
6. Что такое обучение модели	9
6.1. Общая идея	9
6.2. Функция потерь	9
6.3. Градиент	9
6.4. Градиентный спуск	9
6.4.1. Интуиция	10
6.5. Обратное распространение ошибки	10
7. Что такое оптимизатор	10
7.1. SGD	10
7.1.1. Интуиция	10
7.2. Adam	10
7.2.1. Почему он часто хорош	11
7.3. AdamW	11
7.4. Learning rate	11
7.4.1. Если learning rate слишком маленький	11

7.4.2. Если learning rate слишком большой	11
7.4.3. Пример	11
8. Что такое эпоха, батч и итерация	11
8.1. Эпоха	11
8.2. Батч	11
8.3. Итерация	12
8.3.1. Почему не обучают по одному объекту или сразу по всей выборке . .	12
9. Что такое датасет, train, validation и test	12
9.1. Train	12
9.2. Validation	12
9.3. Test	12
9.4. Почему важно не путать эти выборки	12
9.4.1. Пример	12
10. Почему изображения — это особый тип данных	12
11. Что такое свёрточная нейронная сеть	13
11.1. Общая идея CNN	13
11.2. Почему CNN лучше MLP на изображениях	13
11.2.1. 1. Локальность	13
11.2.2. 2. Общие веса	13
11.2.3. Что это даёт	13
12. Что такое свёртка	13
12.1. Определение	13
12.2. Интуиция	14
12.3. Пример	14
12.4. Что такое карта признаков	14
13. Параметры свёртки	14
13.1. Kernel size	14
13.1.1. Что он меняет	14
13.1.2. Пример	14
13.2. Stride	14
13.2.1. Если stride = 1	15
13.2.2. Если stride = 2	15
13.3. Padding	15
13.3.1. Зачем он нужен	15
13.3.2. Пример	15
14. Что такое pooling	15
14.1. Max pooling	15
14.1.1. Пример	15
14.2. Зачем pooling нужен	15
14.3. Интуиция	16
15. Каналы и карты признаков	16

16.	Иерархия признаков в CNN	16
16.1.	Ранние слои	16
16.2.	Средние слои	16
16.3.	Глубокие слои	17
16.4.	Пример для цифр	17
17.	Почему после свёрток часто используют глобальное усреднение	17
17.1.	Почему это удобно	17
17.2.	Интуиция	17
18.	Что такое классификатор на выходе	17
18.1.	Что такое логиты	18
19.	Softmax и вероятности классов	18
19.1.	Что делает softmax	18
19.2.	Пример	18
20.	Cross-entropy loss	18
20.1.	Как это интерпретировать	18
20.2.	Пример	19
20.3.	Почему это удобно	19
21.	Accuracy	19
21.1.	Пример	19
21.2.	Ограничение accuracy	19
22.	Матрица ошибок	19
22.1.	Как читать матрицу	19
22.2.	Пример	19
23.	Переобучение и недообучение	20
23.1.	Что такое недообучение	20
23.2.	Что такое переобучение	20
23.3.	Интуиция	20
23.4.	Пример	20
24.	Что такое регуляризация	20
24.1.	Dropout	20
24.1.1.	Зачем это нужно	20
24.1.2.	Интуиция	21
24.2.	Weight decay	21
25.	Early stopping	21
25.1.	Зачем это нужно	21
25.2.	Параметры	21
25.3.	Пример	21
26.	Гиперпараметры и почему они важны	21
26.1.	Почему они важны	22
26.2.	Пример	22
27.	Что такое гиперпараметрический поиск	22

27.1. Идея	22
27.2. Почему нужна именно validation	22
28. Почему для маленьких изображений параметры нужно выбирать аккуратно .	22
28.1. Пример	23
29. Почему важна нормализация входных данных	23
29.1. Что это даёт	23
30. Почему важен seed	23
31. Что происходит с изображением внутри CNN	23
32. Особенности MNIST	24
33. Особенности FashionMNIST	24
34. Что значит, что сеть «учит признаки»	24
34.1. Пример	24
35. Почему deeper is not always better	25
36. Как читать графики обучения	25
36.1. Loss curves	25
36.2. Accuracy curves	25
36.3. Плато	25
37. Частые ошибки понимания темы	25
37.1. Ошибка 1	25
37.2. Ошибка 2	26
37.3. Ошибка 3	26
37.4. Ошибка 4	26
38. Что нужно уметь объяснить на защите	26
39. Краткое резюме всей темы	26
40. Как лучше учить эту тему	27

1. Зачем нужен этот документ

Этот файл задуман как **полный теоретический конспект** по теме лабораторной работы 4. Его цель — не просто перечислить термины, а собрать в одном месте всю базу, которая нужна, чтобы уверенно понимать тему свёрточных нейронных сетей, обучения моделей на изображениях, подбора гиперпараметров и анализа качества.

Документ полезно читать в двух режимах:

- как учебный конспект перед выполнением лабораторной;
- как материал для повторения перед защитой.

Основной акцент здесь на том, чтобы каждое понятие было объяснено:

- простыми словами;
- формально;
- с интуицией;
- с небольшими примерами.

2. Общая постановка задачи

Лабораторная работа посвящена задаче **классификации изображений**. Это означает, что на вход модели подаётся изображение, а на выходе она должна определить, к какому классу это изображение относится.

В этой лабораторной используются датасеты **MNIST** и **FashionMNIST**.

- В **MNIST** нужно распознавать цифры от 0 до 9.
- В **FashionMNIST** нужно распознавать тип предмета одежды или обуви.

Формально задача выглядит так:

- есть входной объект x — изображение;
- есть истинная метка y — номер класса;
- нужно построить функцию $f(x)$, которая по изображению выдаёт правильный класс.

Если говорить совсем по-простому, то лабораторная отвечает на вопрос: **как научить компьютер смотреть на изображение и понимать, что на нём изображено**.

3. Что такое нейронная сеть

3.1. Базовая идея

Нейронная сеть — это параметризованная функция, которая получает на вход данные и преобразует их через последовательность операций в выходной результат.

Если не углубляться в детали, нейросеть делает примерно следующее:

1. принимает входные числа;
2. умножает их на веса;
3. складывает результаты;
4. применяет нелинейную функцию;
5. повторяет это много раз;
6. в конце выдаёт предсказание.

Главная особенность нейросети не в том, что она умеет считать, а в том, что **она умеет подбирать параметры автоматически** по данным.

3.2. Искусственный нейрон

Один искусственный нейрон можно представить как очень простое вычислительное устройство. Он получает несколько входов $x_1, x_2, \dots, x_n$, у каждого входа есть свой вес $w_1, w_2, \dots, w_n$, а затем считается взвешенная сумма:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b.$$

После этого применяется функция активации:

$$a = \varphi(z).$$

Здесь:

- x_i — входные признаки;
- w_i — веса;
- b — смещение;
- z — линейная комбинация;
- φ — нелинейность;
- a — выход нейрона.

3.2.1. Интуиция

Если объяснять на бытовом уровне, нейрон делает следующее:

- смотрит на несколько входных признаков;
- решает, какие из них важнее, а какие менее важны;
- суммирует их с учётом важности;
- пропускает результат через правило принятия решения.

3.2.2. Маленький пример

Пусть нейрон смотрит на два признака:

- яркость верхней части изображения;
- яркость нижней части изображения.

Если нужно понять, похож ли символ на цифру 1, то нейрон может научиться сильнее реагировать на длинный вертикальный штрих. Тогда один вес станет большим,

другой маленьким, а функция активации поможет превратить эту линейную комбинацию в более удобный сигнал для следующих слоёв.

4. Что такое слой и почему одной линейности недостаточно

4.1. Линейный слой

Если собрать много нейронов вместе, получится **слой**. В матричном виде линейный слой можно записать так:

$$z = Wx + b.$$

Здесь:

- x — входной вектор;
- W — матрица весов;
- b — вектор смещений;
- z — выход слоя до нелинейности.

4.2. Почему нужна нелинейность

Если просто поставить несколько линейных слоёв подряд без функций активации, то они сведутся к одному линейному преобразованию. Это означает, что сеть не станет по-настоящему глубокой по выразительной силе.

Поэтому после линейного слоя нужна функция активации. Она делает модель **нелинейной**, а значит способной описывать сложные зависимости.

4.3. Пример

Представь, что нужно разделить точки двух классов на плоскости.

- Если классы разделяются прямой линией, линейной модели достаточно.
- Если классы образуют круг, спираль или сложную форму, одной прямой уже не хватит.

Именно нелинейности позволяют строить сложные границы решения.

5. Функции активации

5.1. Что это такое

Функция активации — это операция, которая применяется к выходу нейрона после линейного преобразования. Её задача — добавить нелинейность.

5.2. Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Сигмоида переводит любое число в интервал от 0 до 1.

5.2.1. Интуиция

Её удобно интерпретировать как меру уверенности или вероятность.

5.2.2. Плюсы

- гладкая функция;
- удобна для вероятностной интерпретации.

5.2.3. Минусы

- при больших по модулю значениях z функция насыщается;
- градиенты становятся маленькими;
- глубокие сети обучаются хуже.

5.3. Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Эта функция похожа на сигмоиду, но её значения лежат в интервале от -1 до 1.

5.3.1. Интуиция

Она лучше центрирована вокруг нуля, поэтому в некоторых задачах ведёт себя удобнее, чем sigmoid.

5.3.2. Минус

Проблема насыщения и затухающих градиентов всё равно сохраняется.

5.4. ReLU

$$\text{ReLU}(z) = \max(0, z).$$

Это одна из самых популярных функций активации в современных нейросетях.

5.4.1. Интуиция

- если сигнал отрицательный, он отсекается;
- если положительный, он проходит дальше почти без изменений.

5.4.2. Почему ReLU удобна

- очень проста вычислительно;

- не насыщается на положительной полуоси;
- обычно ускоряет обучение глубоких сетей.

5.4.3. Пример

Если после свёртки сеть получила отклик -2.3 , то после ReLU это будет 0 .

Если отклик был 1.8 , то после ReLU останется 1.8 .

Это похоже на правило: «пропускаем только полезные положительные отклики».

6. Что такое обучение модели

6.1. Общая идея

Обучение — это процесс подбора параметров модели по данным так, чтобы предсказания становились лучше.

Для этого нужны три вещи:

- данные;
- функция ошибки;
- алгоритм оптимизации.

6.2. Функция потерь

Функция потерь показывает, насколько плохо модель работает на текущем шаге.

Если модель ошибается сильно, loss большой. Если модель предсказывает хорошо, loss маленький.

То есть обучение можно понимать как задачу минимизации loss.

6.3. Градиент

Градиент показывает, в какую сторону и насколько нужно изменить параметры, чтобы функция потерь уменьшилась быстрее всего.

Если говорить совсем интуитивно, то градиент — это «подсказка», как двигать веса, чтобы модель становилась лучше.

6.4. Градиентный спуск

Базовое правило обновления параметров можно записать так:

$$w_{t+1} = w_t - \eta \nabla L(w_t).$$

Здесь:

- w_t — параметры на текущем шаге;

- η — шаг обучения;
- ∇L — градиент loss по параметрам.

6.4.1. Интуиция

Представь, что модель находится на поверхности холма, а высота — это значение loss. Тогда обучение пытается спуститься вниз, к минимуму.

6.5. Обратное распространение ошибки

Чтобы посчитать градиенты для всех слоёв сети, используется алгоритм **backpropagation** — обратное распространение ошибки.

Идея такая:

- сначала сеть делает прямой проход и считает предсказание;
- затем считается loss;
- потом по правилу цепочки вычисляются производные для каждого параметра от конца сети к началу.

Именно так модель понимает, как менять веса даже в глубоких слоях.

7. Что такое оптимизатор

Оптимизатор — это алгоритм, который получает градиенты и решает, как именно обновлять веса.

7.1. SGD

SGD — stochastic gradient descent. Это базовый оптимизатор.

Он обновляет параметры в направлении антиградиента. Вариант с momentum дополнительно учитывает предыдущие шаги и помогает двигаться устойчивее.

7.1.1. Интуиция

Если обычный градиентный спуск похож на движение маленькими шагами вниз по склону, то SGD с momentum добавляет инерцию.

7.2. Adam

Adam — один из самых популярных оптимизаторов.

Он сочетает:

- идею momentum;
- адаптивный выбор шага для разных параметров.

7.2.1. Почему он часто хорош

- быстро сходится;
- хорошо работает «из коробки»;
- особенно удобен для первых экспериментов.

7.3. AdamW

AdamW похож на Adam, но корректнее работает с weight decay. На практике часто даёт более аккуратную регуляризацию.

7.4. Learning rate

Learning rate — это параметр, который управляет размером шага обновления.

7.4.1. Если learning rate слишком маленький

- обучение идёт медленно;
- модель долго улучшает качество;
- можно не успеть дойти до хорошего решения за ограниченное число эпох.

7.4.2. Если learning rate слишком большой

- обучение становится нестабильным;
- loss может колебаться или расходиться;
- модель может перескакивать через хорошие области.

7.4.3. Пример

Если представить, что ты спускаешься по лестнице в темноте:

- слишком маленький шаг — идёшь очень медленно;
- слишком большой шаг — можно споткнуться и проскочить нужную ступень.

8. Что такое эпоха, батч и итерация

8.1. Эпоха

Эпоха — это один полный проход модели по всей обучающей выборке.

8.2. Батч

Батч — это небольшая группа объектов, которая обрабатывается за один шаг.

Например, если в train 10 000 изображений, а `batch size = 100`, то за одну эпоху будет 100 шагов.

8.3. Итерация

Итерацией часто называют один шаг обновления по одному батчу.

8.3.1. Почему не обучают по одному объекту или сразу по всей выборке

- по одному объекту слишком шумно;
- по всей выборке сразу слишком дорого по памяти и времени;
- батчи дают хороший компромисс.

9. Что такое датасет, train, validation и test

9.1. Train

`train` — это данные, по которым модель реально обучается и обновляет веса.

9.2. Validation

`validation` — это данные, которые нужны для подбора гиперпараметров и контроля качества во время обучения.

Модель на них **не обучается**, а только проверяется.

9.3. Test

`test` — это финальная контрольная выборка. Она используется в конце, когда модель уже выбрана.

9.4. Почему важно не путать эти выборки

Если подбирать модель по `test`, то тест перестаёт быть честной независимой оценкой.

9.4.1. Пример

Если ты много раз смотришь на ответы в контрольной работе и подгоняешь под них решение, то оценка уже не отражает реального знания. С `test` ситуация та же самая.

10. Почему изображения — это особый тип данных

Изображение — это не просто список чисел. В нём важны:

- соседство пикселей;
- локальные структуры;
- границы;
- текстуры;
- композиция более простых признаков в более сложные.

Если развернуть изображение в длинный вектор и подать в обычный MLP, модель не использует пространственную структуру напрямую.

Именно поэтому для изображений особенно важны свёрточные сети.

11. Что такое свёрточная нейронная сеть

11.1. Общая идея CNN

CNN — это нейросеть, специально приспособленная для изображений. Она умеет извлекать локальные признаки и строить иерархию признаков от простых к сложным.

Если говорить образно:

- первые слои видят контуры;
- средние слои видят более сложные фрагменты;
- глубокие слои распознают уже целые характерные паттерны объекта.

11.2. Почему CNN лучше MLP на изображениях

У CNN есть две важные идеи.

11.2.1. 1. Локальность

Фильтр смотрит только на небольшой участок изображения, а не на всё сразу.

11.2.2. 2. Общие веса

Один и тот же фильтр применяется ко всем областям изображения. Это означает, что модель ищет один и тот же признак в разных местах картинки.

11.2.3. Что это даёт

- меньше параметров;
- лучшее использование структуры изображения;
- устойчивость к небольшим сдвигам объекта.

12. Что такое свёртка

12.1. Определение

Свёртка — это операция, при которой маленькое ядро проходит по изображению и вычисляет отклик в каждой позиции.

Формально можно записать это так:

$$y(i, j) = \sum_u \sum_v x(i + u, j + v) K(u, v) + b.$$

Здесь:

- x — входное изображение или карта признаков;
- K — ядро свёртки;
- $y(i, j)$ — значение выходной карты в позиции (i, j) ;
- b — смещение.

12.2. Интуиция

Фильтр можно представить как шаблон, который проверяет: «насколько текущий кусочек изображения похож на нужный паттерн?»

Если похож сильно, отклик большой. Если не похож, отклик маленький.

12.3. Пример

Допустим, есть фильтр, который хорошо реагирует на вертикальную границу. Тогда:

- на вертикальном штрихе цифры 1 он даст большой отклик;
- на пустом фоне отклик будет маленьким.

12.4. Что такое карта признаков

Результат работы одного фильтра — это **карта признаков**.

Если фильтров много, то на выходе слоя получается набор карт признаков. Каждая из них отвечает за свой тип шаблона.

13. Параметры свёртки

13.1. Kernel size

Размер ядра свёртки, например 3x3, 5x5, 7x7.

13.1.1. Что он меняет

- маленькое ядро смотрит на очень локальный контекст;
- большое ядро захватывает более широкий контекст.

13.1.2. Пример

Ядро 3x3 удобно для выделения небольших локальных структур. Ядро 5x5 видит более широкий фрагмент, но и параметров у него больше.

13.2. Stride

Stride — это шаг, с которым фильтр двигается по изображению.

13.2.1. Если `stride = 1`

Фильтр двигается по каждой соседней позиции. Информация сохраняется подробнее.

13.2.2. Если `stride = 2`

Фильтр прыгает через одну позицию. Размер карты признаков уменьшается быстрее, но часть деталей теряется.

13.3. Padding

Padding — это добавление рамки вокруг изображения, чаще всего нулями.

13.3.1. Зачем он нужен

- чтобы управлять размером выходной карты;
- чтобы свёртка не теряла информацию у границ слишком быстро.

13.3.2. Пример

Если использовать ядро 3×3 без padding, изображение будет уменьшаться после свёртки. Если добавить padding, размер можно сохранить.

14. Что такое pooling

Pooling — это операция уменьшения пространственного размера карты признаков.

14.1. Max pooling

Самый популярный вариант — max pooling. Он берёт локальное окно и оставляет максимальное значение.

14.1.1. Пример

Если окно 2×2 содержит значения:

- 0.1
- 0.7
- 0.2
- 0.4

то после max pooling останется 0.7.

14.2. Зачем pooling нужен

- уменьшает размер представления;
- снижает вычислительную стоимость;
- помогает выделять наиболее сильные локальные признаки;
- делает модель устойчивее к небольшим сдвигам.

14.3. Интуиция

Если свёртка отвечает на вопрос «где и насколько сильно найден признак», то pooling оставляет главное и отбрасывает лишнюю детализацию.

15. Каналы и карты признаков

В обычном сером изображении один канал.

В цветном изображении обычно три канала:

- красный;
- зелёный;
- синий.

Но после свёрточных слоёв слово «каналы» уже означает не цвета, а **разные типы признаков**, которые научилась видеть сеть.

Например, после первого блока 16 каналов могут означать:

- детектор вертикальных границ;
- детектор наклонных линий;
- детектор дуг;
- детектор контрастных переходов;
- и так далее.

То есть канал — это одна карта отклика определённого фильтра.

16. Иерархия признаков в CNN

Одна из главных причин успеха CNN — иерархичность признаков.

16.1. Ранние слои

Обычно учат:

- края;
- простые линии;
- контрастные переходы.

16.2. Средние слои

Обычно учат:

- углы;
- дуги;
- комбинации линий.

16.3. Глубокие слои

Обычно учат:

- более сложные фрагменты цифры или объекта;
- характерную форму класса.

16.4. Пример для цифр

Чтобы распознать цифру 8, сеть может пройти такой путь:

- сначала заметить контуры;
- затем заметить две замкнутые области;
- затем собрать из них образ, характерный для 8.

17. Почему после свёрток часто используют глобальное усреднение

В лабораторной после свёрточной части используется `AdaptiveAvgPool2d((1, 1))`.

Теоретически смысл этой операции такой:

- каждая карта признаков сводится к одному числу;
- это число показывает среднюю силу соответствующего признака по всей карте;
- дальше можно подавать эти числа в линейный классификатор.

17.1. Почему это удобно

- не нужно вручную вычислять размер тензора перед `Linear`;
- уменьшается число параметров;
- классификатор становится проще.

17.2. Интуиция

Можно думать так: если один канал отвечает за некоторый признак, то `global average pooling` задаёт вопрос: «насколько вообще этот признак присутствует на изображении?»

18. Что такое классификатор на выходе

После свёрточной части сеть должна выдать оценку для каждого класса.

Обычно на последнем шаге используется линейный слой, который из набора признаков делает набор логитов.

Если классов 10, то на выходе будет 10 чисел.

18.1. Что такое логиты

Логиты — это необязательно вероятности. Это «сырые оценки» уверенности модели по каждому классу.

Чем больше логит правильного класса относительно остальных, тем увереннее модель.

19. Softmax и вероятности классов

Чтобы превратить логиты в вероятности, часто используют `softmax`:

$$p_c = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

Здесь:

- z_c — логит класса c ;
- p_c — вероятность класса c .

19.1. Что делает softmax

- все выходы становятся неотрицательными;
- сумма вероятностей становится равной 1;
- можно интерпретировать результат как распределение вероятностей по классам.

19.2. Пример

Пусть логиты равны:

- для класса 3: 4.2
- для класса 5: 1.1
- для остальных меньше

Тогда после softmax вероятность класса 3 будет самой большой.

20. Cross-entropy loss

Для многоклассовой классификации часто используют **кросс-энтропию**.

Если истинный класс равен y , то loss для одного объекта можно понимать как:

$$L = -\log p_y.$$

20.1. Как это интерпретировать

- если модель дала большую вероятность правильному классу, loss маленький;
- если модель дала маленькую вероятность правильному классу, loss большой.

20.2. Пример

Если правильный класс имеет вероятность 0.9, то loss небольшой.

Если правильный класс имеет вероятность 0.1, то loss намного больше.

20.3. Почему это удобно

Кросс-энтропия заставляет модель не просто угадывать индекс класса, а становиться уверенной именно в правильном классе.

21. Accuracy

Accuracy — это доля правильно классифицированных объектов.

Формально:

$$\text{accuracy} = \frac{\text{число верных ответов}}{\text{число всех объектов}}.$$

21.1. Пример

Если из 1000 изображений модель правильно классифицировала 930, то accuracy равна 0.93 или 93%.

21.2. Ограничение accuracy

Accuracy — очень удобная метрика, но не всегда достаточная. Она не показывает, какие именно классы чаще всего путаются.

Поэтому полезно смотреть и на confusion matrix.

22. Матрица ошибок

Матрица ошибок показывает, как модель путает классы.

22.1. Как читать матрицу

- строки — истинные классы;
- столбцы — предсказанные классы.

Если модель работает хорошо, то большие значения должны лежать в основном на главной диагонали.

22.2. Пример

Если класс `Shirt` часто попадает в `T-shirt`, то в соответствующей ячейке матрицы будет большое число. Это говорит не только о качестве, но и о характере ошибок.

23. Переобучение и недообучение

23.1. Что такое недообучение

Недообучение означает, что модель слишком слабая или обучена недостаточно.

Признаки:

- низкий `train accuracy`;
- низкий `validation accuracy`;
- модель плохо схватывает даже обучающую закономерность.

23.2. Что такое переобучение

Переобучение означает, что модель слишком хорошо подстроилась под `train` и хуже обобщает на новые данные.

Признаки:

- `train accuracy` высокий;
- `validation accuracy` заметно ниже;
- разрыв между `train` и `validation` увеличивается.

23.3. Интуиция

Недообучение — модель не научилась.

Переобучение — модель научилась слишком буквально запоминать `train`, а не общую закономерность.

23.4. Пример

Если на `train` модель имеет 99%, а на `validation` только 85%, это сильный сигнал переобучения.

24. Что такое регуляризация

Регуляризация — это набор приёмов, которые помогают модели не переобучаться.

24.1. Dropout

Dropout случайно зануляет часть признаков во время обучения.

24.1.1. Зачем это нужно

- модель не может слишком сильно полагаться на один узкий набор признаков;
- признаки становятся более устойчивыми;
- обобщение обычно улучшается.

24.1.2. Интуиция

Если ученик всегда решает задачу одним и тем же шаблоном, он становится хрупким к небольшому изменению условия. Dropout заставляет модель учиться более надёжными стратегиям.

24.2. Weight decay

Ещё один способ регуляризации — штрафовать слишком большие веса. Это помогает делать модель более «сдержанной».

25. Early stopping

Early stopping — это остановка обучения, если качество на validation перестало улучшаться.

25.1. Зачем это нужно

- не тратить лишнее время;
- не уходить слишком далеко в переобучение;
- сохранить лучшую версию модели.

25.2. Параметры

- `patience` — сколько эпох ждать без улучшения;
- `min_delta` — насколько должно улучшиться качество, чтобы считать это реальным прогрессом.

25.3. Пример

Если `patience = 5`, это означает: если 5 эпох подряд validation не стала лучше заметно, обучение останавливается.

26. Гиперпараметры и почему они важны

Гиперпараметры — это настройки, которые не учатся автоматически, а задаются человеком.

В этой лабораторной к ним относятся:

- тип блока;
- число каналов;
- размер ядра;
- `stride`;
- параметры `pooling`;
- `learning rate`;
- `batch size`;

- число эпох;
- patience.

26.1. Почему они важны

Они определяют:

- насколько сложной будет модель;
- насколько быстро она будет учиться;
- насколько вероятно переобучение;
- сколько времени займёт обучение.

26.2. Пример

Если увеличить число каналов, модель сможет извлекать больше разных признаков, но число параметров вырастет, а значит увеличатся время обучения и риск переобучения.

27. Что такое гиперпараметрический поиск

Когда мы пробуем несколько архитектур и несколько значений learning rate, мы фактически выполняем **поиск по сетке гиперпараметров**.

27.1. Идея

Есть несколько списков вариантов. Мы перебираем комбинации и сравниваем качество на validation.

27.2. Почему нужна именно validation

Потому что поиск должен выбирать модель по данным, которые модель не видела при обучении, но которые ещё не являются тестом.

28. Почему для маленьких изображений параметры нужно выбирать аккуратно

В MNIST и FashionMNIST изображения имеют размер 28x28. Это очень маленький размер по меркам компьютерного зрения.

Поэтому:

- слишком агрессивные stride могут быстро уничтожить детали;
- слишком большие ядра не всегда полезны;
- слишком глубокая и широкая модель может быть избыточной.

28.1. Пример

Если несколько раз подряд сильно уменьшать размер карты признаков, то модель может потерять структуру изображения ещё до того, как извлечёт полезные признаки.

29. Почему важна нормализация входных данных

Нормализация переводит входные значения к более удобному масштабу.

Формула стандартной нормализации выглядит так:

$$x' = \frac{x - \mu}{\sigma}.$$

Здесь:

- μ — среднее;
- σ — стандартное отклонение.

29.1. Что это даёт

- более стабильное обучение;
- лучшее поведение градиентов;
- более предсказуемую работу оптимизатора.

30. Почему важен seed

Seed нужен для воспроизводимости эксперимента.

Если фиксировать seed, то:

- одинаково инициализируются веса;
- одинаково выбираются подвыборки;
- одинаково перемешиваются данные.

Это не делает результаты абсолютно идентичными во всех средах, но сильно повышает сопоставимость запусков.

31. Что происходит с изображением внутри CNN

Очень полезно мысленно представить путь одного изображения через сеть.

1. Изображение загружается как тензор.
2. При необходимости нормализуется.
3. Поступает в первый свёрточный блок.
4. Фильтры выделяют локальные признаки.
5. ReLU убирает отрицательные отклики.
6. Pooling уменьшает размер карты и оставляет сильные сигналы.

7. Следующие блоки строят всё более сложные признаки.
8. Global average pooling сводит каждую карту к одному числу.
9. Линейный классификатор выдаёт логиты классов.
10. Softmax и cross-entropy оценивают, насколько модель права.

Если держать в голове именно эту цепочку, теория CNN становится гораздо понятнее.

32. Особенности MNIST

MNIST — это сравнительно простая задача.

Почему она простая:

- изображения маленькие;
- фон относительно чистый;
- объект обычно расположен по центру;
- классы в среднем хорошо различимы.

Поэтому на MNIST даже относительно небольшая CNN может показать очень хорошую ассигасу.

33. Особенности FashionMNIST

FashionMNIST устроен формально так же, но задача обычно сложнее.

Почему:

- разные предметы одежды визуально похожи;
- форма объектов пересекается сильнее;
- некоторые классы трудно различимы даже для человека без внимания.

Поэтому FashionMNIST обычно полезнее как датасет для сравнения архитектур и регуляризации.

34. Что значит, что сеть «учит признаки»

Когда говорят, что нейросеть учит признаки, это означает, что её фильтры и веса постепенно начинают реагировать на полезные закономерности в данных.

Это не ручное выделение признаков, а автоматическое. Мы не говорим модели заранее: «вот этот контур важен». Вместо этого мы даём данные и loss, а модель сама подстраивает фильтры так, чтобы улучшать предсказания.

34.1. Пример

Для цифры 7 модель может научиться реагировать на:

- горизонтальную верхнюю черту;

- диагональный штрих вниз.

Для θ — на круглую замкнутую форму.

35. Почему deeper is not always better

Интуитивно может казаться, что чем глубже сеть, тем лучше. На практике это не всегда так.

Причины:

- больше параметров;
- дольше обучение;
- выше риск переобучения;
- на маленьких задачах слишком глубокая сеть может быть просто не нужна.

Поэтому подбор архитектуры всегда должен быть эмпирическим, а не строиться только на идее «побольше значит лучше».

36. Как читать графики обучения

36.1. Loss curves

Если `train loss` и `val loss` снижаются вместе, это хороший знак.

Если `train loss` падает, а `val loss` начинает расти, это сигнал переобучения.

36.2. Accuracy curves

Если `train accuracy` и `val accuracy` растут и остаются близкими, модель обобщает неплохо.

Если `train accuracy` почти идеальна, а `val accuracy` заметно ниже, модель переобучается.

36.3. Плато

Если графики почти не меняются, модель вышла на плато. Это может означать:

- `learning rate` слишком маленький;
- модель упёрлась в предел своей выразительности;
- данные уже хорошо выучены.

37. Частые ошибки понимания темы

37.1. Ошибка 1

Думать, что CNN — это просто «ещё одна нейросеть».

На самом деле CNN специально устроена под пространственную структуру изображений.

37.2. Ошибка 2

Считать, что высокая train ассурасу автоматически означает хорошую модель.

Нет. Главное — как модель работает на новых данных.

37.3. Ошибка 3

Считать, что больше параметров всегда лучше.

Нет. Нужно учитывать размер задачи, риск переобучения и вычислительную цену.

37.4. Ошибка 4

Путать роль validation и test.

Validation нужна для выбора модели, test — для финальной независимой оценки.

38. Что нужно уметь объяснить на защите

Если ты хочешь уверенно защитить лабораторную, желательно уметь чётко объяснить следующие вещи.

1. Что такое нейрон, слой и функция активации.
2. Почему для изображений лучше использовать CNN, а не простой MLP.
3. Что делают ядро свёртки, stride и padding.
4. Что делает max pooling и зачем он нужен.
5. Что такое карта признаков и канал.
6. Что такое logits, softmax и cross-entropy.
7. Чем отличаются train, validation и test.
8. Что такое переобучение и как его увидеть по графикам.
9. Для чего нужны dropout и early stopping.
10. Почему подбор гиперпараметров идёт по validation, а не по test.

39. Краткое резюме всей темы

Вся тема лабораторной сводится к следующей логике.

- Изображение содержит локальные структуры.
- CNN умеет извлекать эти структуры через свёртки.
- Сеть строит иерархию признаков от простых к сложным.

- Обучение происходит через минимизацию loss с помощью градиентов и оптимизатора.
- Качество оценивается не только по train, но и по validation и test.
- Чтобы модель не переобучалась, используют регуляризацию и раннюю остановку.
- Чтобы найти хорошую архитектуру, перебирают гиперпараметры.

Если эта логика понятна, значит теоретическая база лабораторной уже собрана правильно.

40. Как лучше учить эту тему

Самый эффективный порядок повторения такой:

1. Сначала понять, как работает обычный нейрон и обучение через loss и градиенты.
2. Потом понять, почему изображения требуют особой архитектуры.
3. Затем отдельно разобрать свёртку, pooling и карты признаков.
4. После этого разобрать softmax, cross-entropy и accuracy.
5. И только затем переходить к вопросам переобучения, dropout, early stopping и подбора гиперпараметров.

Если хочешь, следующим шагом я могу сделать ещё один файл: `main_exam.typ` с форматом «вопрос -> короткий, но сильный ответ для защиты», чтобы по нему быстро готовиться перед сдачей.